

Week 01:

从 Demo 到上线AI 为什么不能直接用？

先立标准，再谈智能

曾丹



开场定性 · 企业 AI 第一条生死线

为什么你的 AI Demo 一上生产就翻车？

从 3 个真实事故，看懂企业 AI 的第一道生死线

这不是一节事故八卦课，而是整门实战营的第一讲：先拆掉 Demo 幻觉，再建立 AI 系统交付视角。

现实冲突

三案拆解

工程动作

Lesson02 过桥

一句话结论

本讲唯一主判断

能演示，只说明局部成立。

能上线，要求链路可控。

后面整门课不是在堆技术点，而是在把这条交付链补完整。



为什么今天必须先讲这件事

企业现场的真实冲突

- AI 采用率已高，但从试点走向规模化交付仍然困难。
- 用户不会区分官网答复、机器人答复、人工答复。
- 一旦进入权限、PII、动作和责任语境，系统就换了世界。

这一讲要先立住什么

- 事故最先从哪一层开始？
- Demo 阶段为什么没有暴露？
- 如果你是架构师，第一反应该补哪一层？



三个事故，先并列再拆解

Air Canada

错误政策答复

旅客信了官网 bot 的票务说明，最后企业要为错误信息负责。

NYC MyCity

官方答复越权

公共服务场景里，错误建议会直接升级成治理和合规问题。

DoNotPay

能力宣称越界

不是某次答错，而是产品把能力主张推进到了监管边界之外。



三条新闻，背后其实是同一种工程问题

表面看

是三种完全不同的失败

实际上

都是把不受控的能力放进了正式服务链路

架构师视角

先找首层失控，再找放大层，最后再定工程动作



Air Canada: 不是 bot 答错一句话, 而是企业要为它负责

故事链

1. 旅客在官网询问丧葬票政策
2. bot 给出一套看起来完整的执行建议
3. 用户按它的建议购票和申请
4. 企业后续不能再把责任推给“只是机器人答错”

你先记住

- 官网里的 AI 答复会被用户当成正式服务口径
- 一旦进入赔付或规则语境, 错误回答会升级成责任问题
- 风险不在“像不像人”, 而在“证据和边界有没有一起上线”



Air Canada: 架构师拆解

表面事故

政策答复错误，旅客据此行动

首层失控

数据 / 知识层没有把规则来源和版本约束住

放大层

生成层把不可靠内容包装成可信答复

工程动作

补来源约束、引用、拒答边界和人工升级路径



MyCity：一旦挂上官方入口，错误建议就会升级成治理问题

故事链

1. 官方 AI 服务被放进公共服务入口
2. 媒体和研究者用真实问题持续提问
3. 系统给出可能违法或不合规的建议
4. 问题迅速升级成公共治理与信任问题

你先记住

- 公共服务语境里，错误不是普通产品 bug
- 风险来自错误答复带着官方权威感
- 真正缺的是高风险拒答、来源白名单和治理边界



MyCity: 架构师拆解

表面事故

官方 bot 给出错误甚至潜在违法建议

首层失控

检索层没有把法规来源和边界约束住

放大层

治理 / 观测层没有把高风险场景拦下来

工程动作

来源白名单、高风险拒答、升级路径、审计可见



DoNotPay：问题不只是答错，而是产品把自己讲过界了

故事链

1. 产品长期往“AI lawyer”方向包装
2. 用户规模和关注度上来后，监管开始追问
3. 问题变成系统是否真的具备这类高风险能力
4. 事故最终表现为能力宣称和责任边界长期错位

你先记住

- 企业 AI 不一定先从回答翻车
- 很多系统先从能力主张翻车
- 只要宣称超出真实能力，治理和责任问题会先爆



DoNotPay: 架构师拆解

表面事故

产品主张越过了监管和专业服务边界

首层失控

工具 / 动作层没有划清能力与不可做动作

放大层

治理 / 观测层没有把宣称、证据和责任绑定

工程动作

收紧能力主张，限制动作，补人工复核和发布治理



从新闻回到方法：固定用同一套模板复盘

第一步

看清表面事故

第二步

判断哪一层先失控

第三步

判断哪一层把伤害放大

第四步

再决定工程动作



五层失控模型：先别猜参数，先看哪一层裂开了

事故定位主图

五层失控模型

When AI breaks, which layer cracked?

事故
↓
逐层
排查

① 数据层

输入数据 · 知识 · 版本 · 来源 · 权限边界

② 检索层

能否稳定召回正确证据, 而非只召回“像相关”的内容

③ 生成层

模型如何组织答案 · 何时拒答 · 不确定性是否被包装成确定性

④ 工具/动作层

查数据 · 执行动作 · 调外部系统 · 动作硬边界

⑤ 治理/观测层

评测 · 日志 · HITL · 回滚 · 用户通知 · 责任归属

每层都是防线,
任何一层裂开 = 失控

固定五步复盘动作

01 补背景

先补事实和责任
语境，不要上来
就猜技术原因

02 看表面事故

先把外部看到的
失败讲清楚

03 找首层失控

判断到底是哪一
层先裂开

04 找放大层

判断是哪一层把
伤害放大

05 定工程动作

最后才决定该补
什么、拦什么、
改什么

先定层，再定动作。顺序不能倒。



从诊断到处方：后面的课程都在补这张图

课程映射图



Lesson01 先把诊断工具立住；后面的周次再按层补齐输入边界、检索证据、工具动作、评测观测和治理发布。

一旦接受事故不是偶然，就必须区分两个世界

Demo 世界和生产世界 不是同一个世界

前者允许人肉补上下文，后者要求系统把证据、边界、观测和回滚一起补上。



Demo 世界 vs 生产世界：先看四个维度

数据

- Demo：静态样例、手工准备、无版本差异
- 生产：持续更新、口径漂移、权限和 PII

用户

- Demo：提问友好、上下文完整
- 生产：问题歧义、分布极广、会撞边界

系统

- Demo：单人掌控、单路径运行
- 生产：跨系统协作、高并发、失败需系统处理

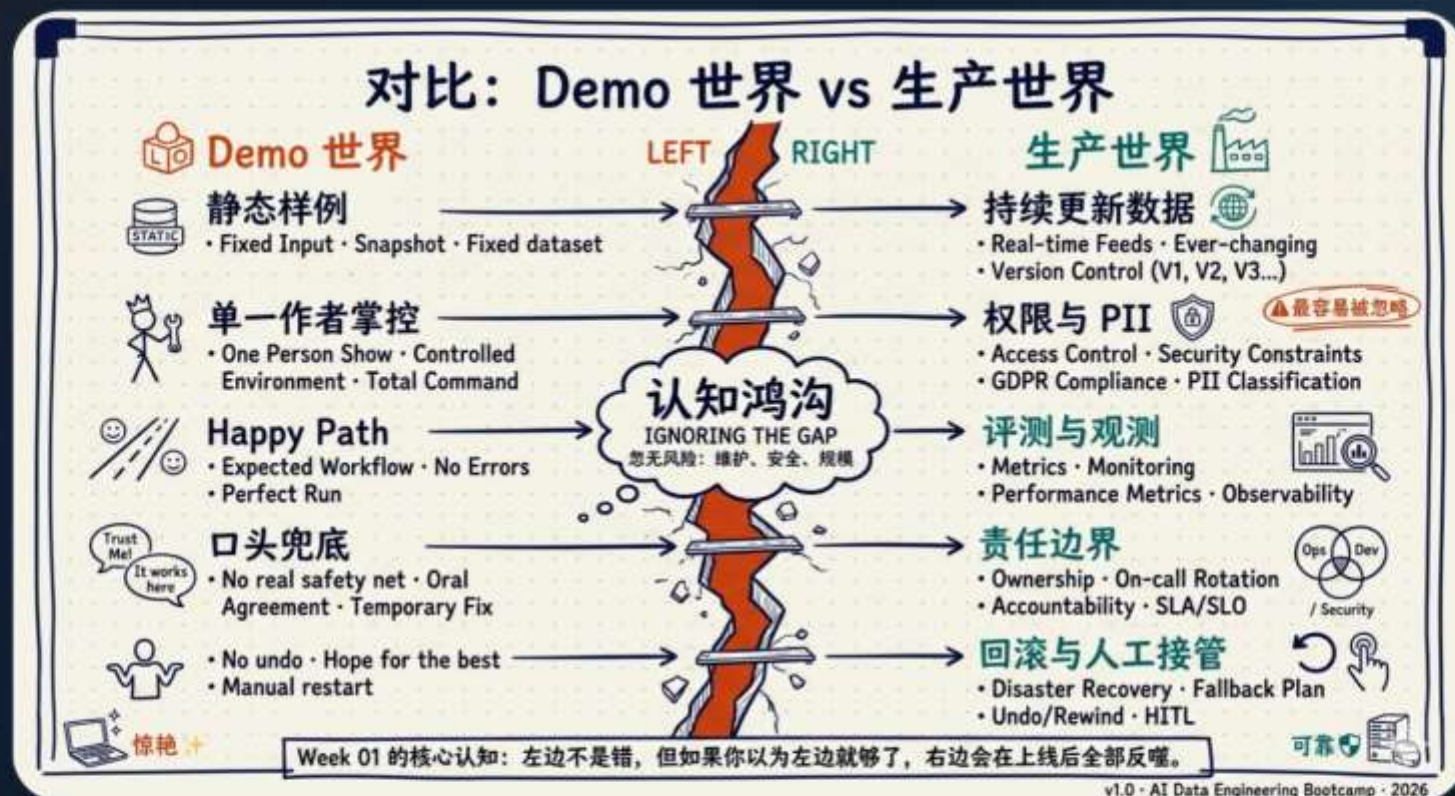
治理

- Demo：先做出来再说
- 生产：评测、观测、责任边界、发布回滚都要前置



Demo 世界 vs 生产世界：认知鸿沟长这样

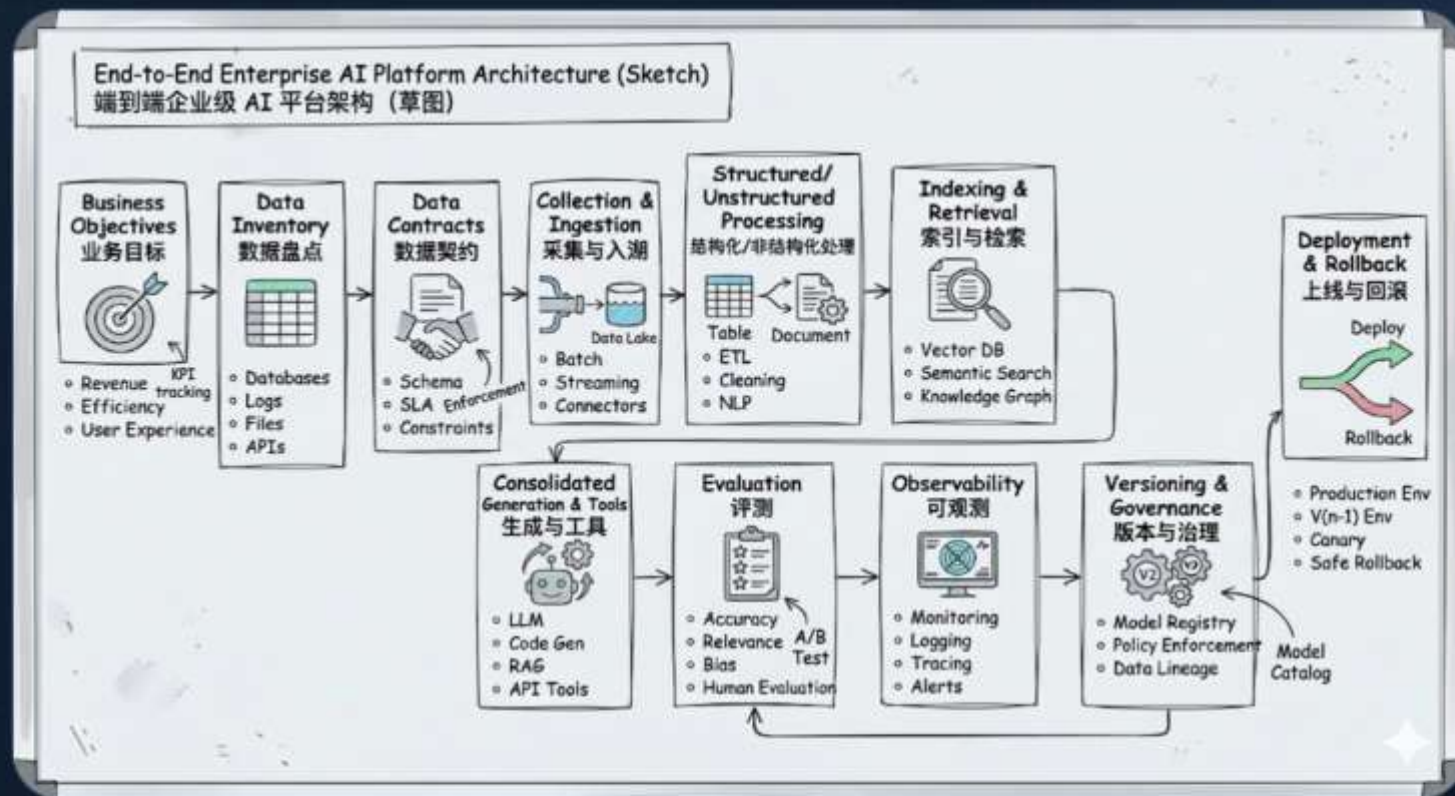
认知转轴图



左边是 Demo 阶段常见的隐藏假设，右边是进入生产后必须面对的真实约束。

企业 AI 从 0 到 1 的交付链

全课主链



后面整门课不是 15 个散点，而是在把这条固定主链补齐。

真正要记住的不是名词，而是顺序

业务目标

先定义要解决什么真实问题

数据盘点 → 数据契约 → 采集与入湖 → 索引与检索 → 生成与工具 → 评测与观测 → 版本治理 → 上线与回滚

顺序一乱，后面就会变成边做边返工；顺序站住，后面的每一周才有工程意义。



主案例不是 FAQ bot，而是企业支持系统原型

OmniSupport Copilot 是什么

- 回答帮助中心、FAQ、Release Notes、API 文档中的规则问题
- 读取工单、评论线程、状态字段等实时业务事实
- 在受控前提下执行查询、创建、更新等动作
- 全程带证据引用、权限约束、审计字段与回滚意识

这一讲为什么要先把它拉进来

- 把知识、事实、权限、动作、治理同时放进一个系统
- 它不是课堂 Demo，而是整门课的工程基线
- 后面 14 周都围绕它补链，不再另起项目



这个案例一上来就同时牵出五类约束

规则知识

FAQ、说明文档、Release Notes 到底怎么解释

实时事实

工单状态、评论线程、历史处理记录是否一致

权限边界

当前提问者到底能看到什么、不能看到什么

动作边界

系统只能建议，还是可以真正创建 / 更新工单

人工介入

高风险、高不确定性或越权时要在哪停下来

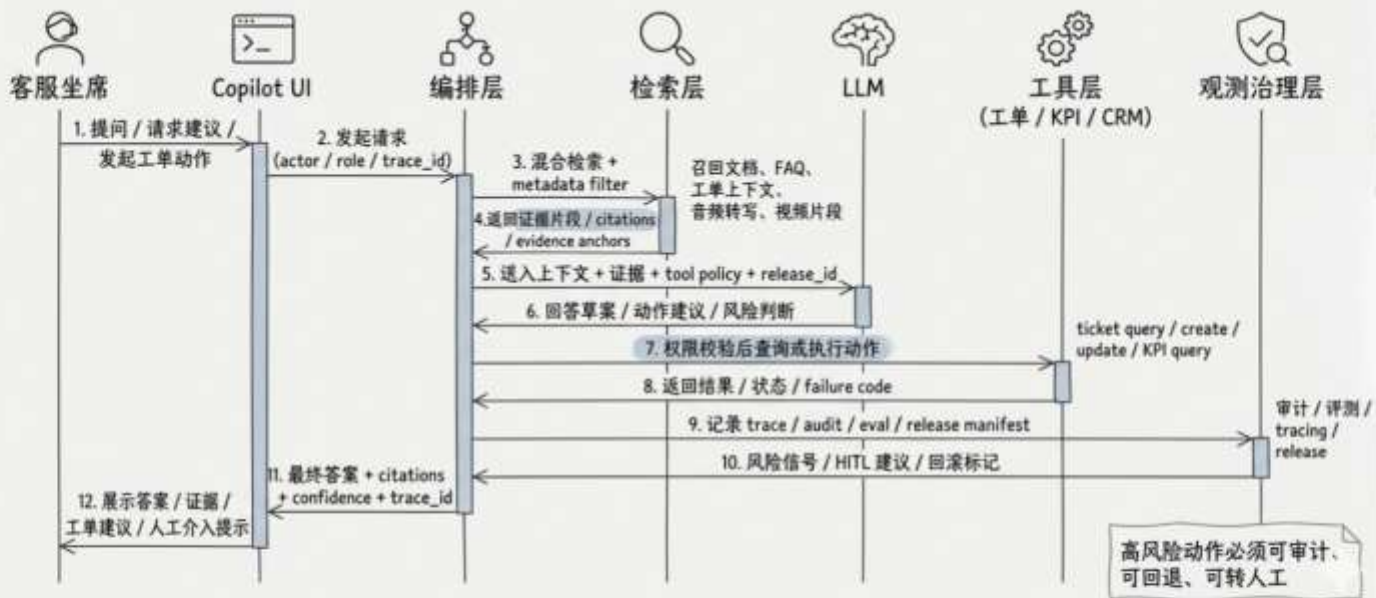


一个“简单问题”的背后，其实是一条受控请求链

受控请求链

主案例：OmniSupport Copilot 客服知识库 + 工单联动

一个看似简单的问题，背后同时牵出：多模态知识、证据引用、权限校验、动作边界、人工介入、审计追踪、版本回滚



知识、检索、生成、工具与审计不是五个点，而是一条必须协同受控的请求链。

从 Week01 开始，你要把它看成同一个工程基线

方法论验证器

前面讲的事故、边界、交付链都要压回到这个案例

统一练习场

后面 14 周都围绕它继续补能力，而不是换项目

唯一工程基线

项目骨架、权限、PII、HITL、风险边界都从这里开始立



今天行业真正收敛的，不是再调一轮 Prompt

评测要前置

不能等出事以后才知道系统有没有变差

观测要前置

请求、证据、动作和失败都要能看见

风险边界要前置

权限、PII、动作和责任边界要先立

能力主张要和证据绑定

能说什么、敢说什么，必须和真实能力一致



会演示 vs 可上线

维度	会演示	可上线
数据	静态样例	有版本、更新和口径边界
检索	能召回一点内容	证据稳定、可解释、可追溯
输出	看起来像能答	有拒答、置信与风险边界
权限	默认无边界	用户、租户、PII 分层明确
工具	演示动作可跑	参数受控、动作可审计
评测	靠人工感觉	有指标、用例和回放
观测	不出错就算好	请求、证据、动作、失败可观测
回滚	出事再说	有降级、停机和回退路径
成本	不太关心	延迟、推理和资源成本可管理
责任边界	口头说明	owner、升级路径和治理边界清楚

课堂版 Launch Readiness

先问有没有这 6 类能力

- 数据版本和更新边界
- 权限与 PII 分层
- 证据引用与来源约束
- 工具动作和参数约束
- 评测与观测
- 回滚、HITL 与责任边界

分数低，不代表项目没价值

- 只说明它还停在 Demo 阶段
- 也说明后面课程有明确的补课顺序
- Lesson02 会先从 Done 和签字标准开始



最后只收三条 takeaways

- 01 AI 翻车通常不是模型单点失误，而是链路共同失控。
- 02 后面 15 周是在补企业 AI 的交付链，而不是堆技术点。
- 03 下一讲必须先定义 Done 和验收标准，而不是先选工具。



交付标准不是感觉题

LESSON02 · 可签字交付

知识库问答 + 工单联动， 什么时候才算真正 Done?

这一讲不是继续追求“更像人回答”，而是先把业务、架构、合规、运维四个角色都能签字的条件讲清楚。

从这一讲开始，课程主线正式从“事故诊断”转入“交付标准”。后面 14 周学的每一层能力，都是在给这些签字条件补证据。

开场冲突

五层 Done

签字现场

工件落地

一句话判断

本讲唯一主判断

**Done 不是“能回答”，而是
业务敢验收、质量能门禁、安全有边界、运行可降级、治理可追
责。**

只要这五层里还有一层站不住，项目就还停留在“能演示”，没有进入“可交付”。



这节课的课堂地图：先立判断入口

开场冲突

- 把“能答”的系统放进上线前评审会
- 看懂为什么现场顺，不等于任何角色敢放行

判断框架

- 用 Demo vs Production
- 用五层 Done，把“做完了没”改成可签字判断



这节课的课堂地图：再看签字现场和工件

签字现场

- 业务 / 架构 / 合规 / 运维进入同一张签字板
- 看他们为什么会拦你

三张开工模板

- 留下 3 张能直接改成项目文档的模板
- 不是只记几个观点，而是先把表写出来



开场冲突：为什么“能答”仍不能上线

POC 阶段看起来已经成功

- 能回答账户冻结、额度调整、账单问题等常见 FAQ
- 能从知识库生成解释
- 能调用工单接口发起申请
- 业务方会说：“看起来已经能用了”

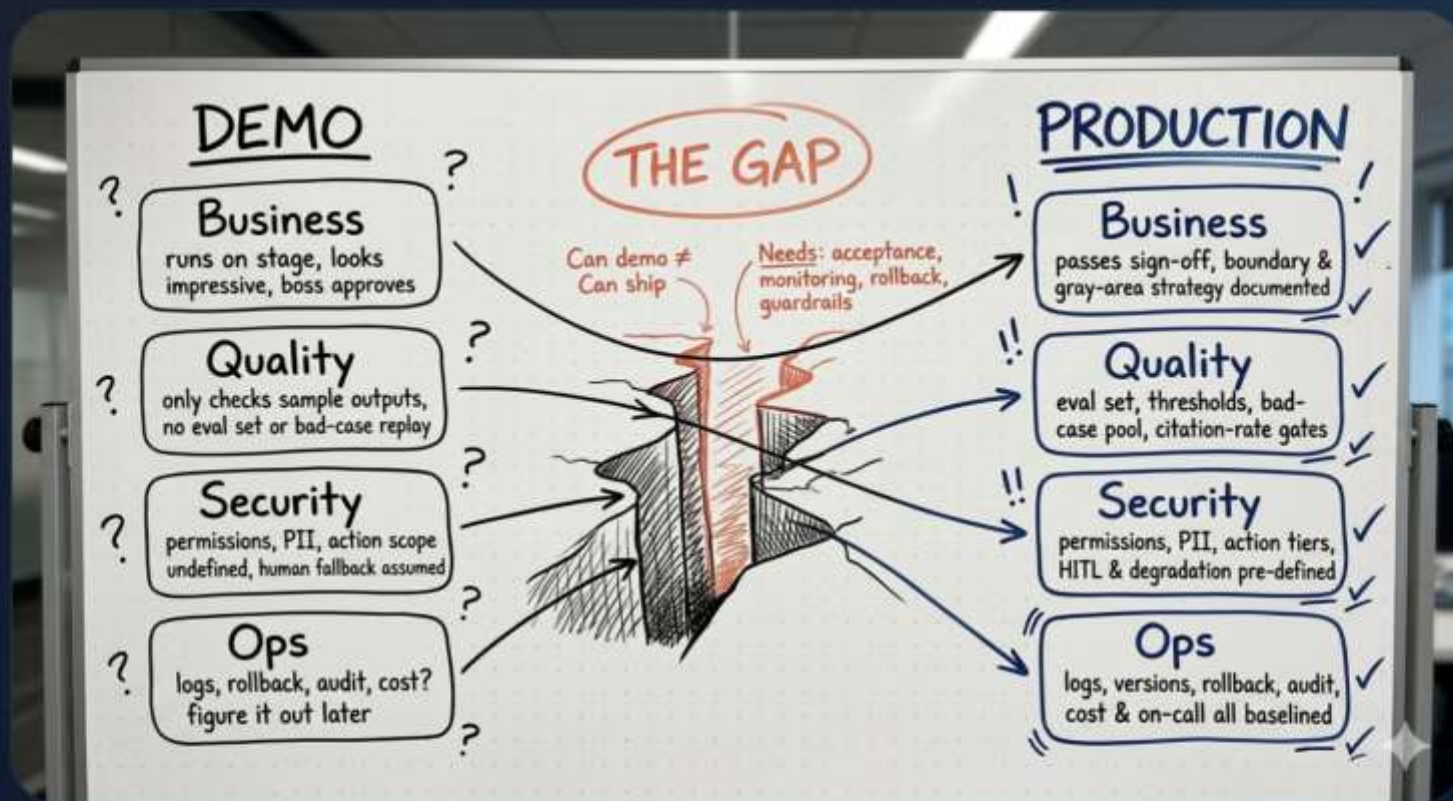
一进试点就没人敢签字

- 回答引用旧版本业务规则
- 没有按用户等级区分适用建议
- “建议建单”和“自动建单”没有分层
- 缺少证据锚点，主管无法复核
- 坏例无法精确回放



Demo vs Production: 中间差的是签字条件

认知转轴图



从 Demo 到 Production, 中间差的不是一点工程细节, 而是业务、质量、安全、运行、治理五类签字条件。

先把判断翻过来：Done = 可签字交付

认知翻转

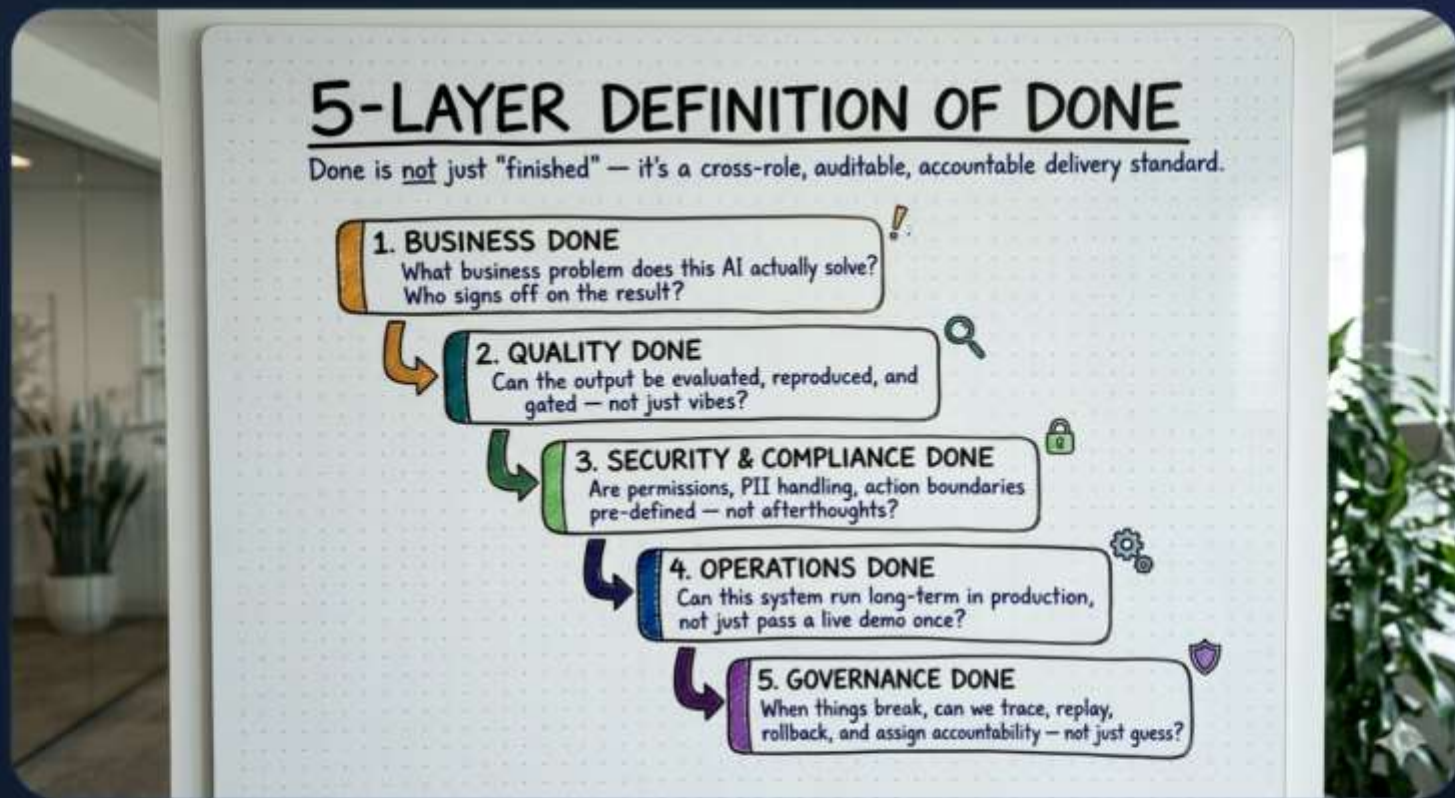
**技术团队不该再问“现在能不能答”，
而要先问“今天上线谁会拦我”。**

真正的交付判断，不是系统会不会说，而是业务、架构、合规、运维四个角色有没有同时获得可签字证据。



五层 Done：这五层要一起站住

交付标准总图



企业 AI 的 Done 从来不是单条效果指标，而是业务、质量、安全、运行、治理五层标准同时站住。

五层 Done：先看前三个签字问题

01 业务层

- 改善什么业务结果
- 谁来签字
- 灰度范围和失败容忍度

02 质量层

- 评测集 / 坏例池
- 阈值和门禁
- 变更后能否稳定回归

03 安全层

- PII 和权限边界
- 动作分级是否前置
- 哪些动作必须进 HITL

先看前三层

业务、质量、安全先站住，系统才有资格继续谈运行和治理。

五层 Done：再补后两层，系统才算能负责

04 运行层

- 失败时能否降级
- 告警和回滚是否存在
- 成本、延迟是否有目标

05 治理层

- 数据 / 索引 / Prompt / 评测版本
- 上线与回滚链路
- 出问题后能否追责

一句话收口

只要这五层里还有一层站不住，就不能把“能答”当成“Done”。

后面 14 周不是给系统堆功能，而是在给这 5 个签字问题逐层补证据。

四角色签字板：先看业务和架构为什么会拦你

业务方

看结果

先问什么

- 目标场景和成功指标是什么
- 灰度范围和失败容忍度是什么

如果没有证据

- 看起来有用，也不会直接验收

架构方

看链路

先问什么

- 数据、索引、Prompt、工具能否持续演进
- 契约、版本、回放链路是否稳定

如果没有证据

- 系统能跑，也不给生产准入



四角色签字板：再看合规和运维为什么会拦你

合规 / 风控

看边界

先问什么

- PII、权限、动作分级是否前置
- 哪些必须拒答，哪些必须 HITL

如果没有证据

- 人工兜底没写成规则，就不会放行

运维 / SRE

看兜底

先问什么

- 故障能不能定位、降级、回滚
- 健康阈值和告警规则是什么

如果没有证据

- 演示越顺，运行时越没人敢兜底



OmniSupport 一进评审会：先看业务和架构在看什么

业务

先问效果

第一句追问

- 首问解决率和建单负担真的会改善吗

真正证据

- 目标场景、成功指标、试点范围

不满足时

- 延后上线，先补验收口径

架构

先问可演进

第一句追问

- 数据、索引、Prompt、工具链能不能持续演进

真正证据

- 版本、契约、可回放链路

不满足时

- 不给生产准入

OmniSupport 一进评审会：再看合规和运维在看什么

合规

先问边界

第一句追问

- 哪些问题必须拒答，哪些动作必须 HITL

真正证据

- PII 规则、权限边界、证据引用要求

不满足时

- 要求补边界后再评审

运维

先问兜底

第一句追问

- 失败时怎么降级，变更后怎么回滚

真正证据

- 健康检查、告警、降级与回滚策略

不满足时

- 只能灰度，不允许放量

三个课堂工件：最后落成三张表

工件 1

交付标准记分卡

- 先把场景、目标、指标、状态写清
- 先对齐签字口径
- 它是后续工程动作的上游

工件 2

风险边界清单

- 把边界写成系统开关
- 它是动作分级入口

工件 3

质量门禁表

- 把指标、阈值、评测集、回退动作写出来
- 从“看起来不错”切到“门禁过没过”



模板 1：先把交付标准记分卡写起来

交付标准记分卡

template scene / goal / metrics / status

- 先把“签什么”写清楚
- 再谈后面的工程动作

```
1 scene: "客服问答 + 建单联动"
2 business_goal: "减少重复查询"
3 success_metrics:
4   - "FAQ 首问解决率"
5   - "建议建单采纳率"
6 status: "可灰度"
```



模板 2：再把风险边界清单写出来

风险边界清单

template 适用用户 / 禁止场景 / 证据要求 / 禁止自动执行

- 先把系统行为边界写成规则
- 它不是法务附录，而是系统行为开关

- 1 ## 业务验收口径 & 风险边界清单
- 2 - 适用用户：客服 / 值班主管
- 3 - 禁止场景：高风险账户操作
- 4 - 证据要求：制度解释 / 规则口径
- 5 - 禁止自动执行：额度修改



模板 3：最后把质量门禁表补齐

质量门禁表

template metric / threshold / eval_dataset / fallback_action

- 把“看起来不错”切成“门禁过没过”
- 把失败后的 fallback 也一起写清

```
1 metric: "FAQ 首问解决率"  
2 threshold: ">= 72%"  
3 eval_dataset: "客服坏例池 v1"  
4 fallback_action: "降级到知识检索"
```



为什么后面 14 周都在给 Done 补证据

后续周次	会补哪类证据	对应 Lesson02 的哪一层 Done
Week02	输入范围、字段契约、门禁规则	业务层 / 安全与合规层
Week03	采集、入湖、原始数据可回放	质量层 / 治理层
Week08	检索服务与 API 交付	质量层 / 运行层
Week10	工具层、动作边界、HITL	安全与合规层 / 运行层
Week12	Tracing、观测、问题定位	运行层 / 治理层
Week14	发布、回滚、版本治理	治理层

一句话收口

**后面 14 周不是在给系统堆功能，
而是在给这 5 层签字条件补证据。**

这个认知先立住，后面的路线判断和工程基线才会真正有意义。

路线先于技巧：先回答企业 AI 该走哪条路

LESSON03 · 路线判断

别一上来就写 RAG：企业级 AI 到底该走哪条数据工程路线？

这一讲不比谁的 Prompt 更会写，也不先讲哪种检索更高级。它只做一件事：把学员从“功能能演示”，拉到“路线能上线”。

如果路线判断错了，后面加的每一层能力都会变成补救；如果路线判断先立住，后面 14 周就是在按顺序补链。

事故开场

最短路径

目标架构

阶段演进

一句话判断

本讲唯一主判断

**企业 AI 的主问题从来不是
“模型够不够聪明”，而是“路线是不是一开始就走对了”。**

后面所有关于数据、检索、工具、评测和治理的能力，都是路线判断之后才有意义的工程层。



这节课只回答三个问题

问题 1

最短路径

为什么脚本式
RAG 很快，但
迟早会撞生产
墙？

问题 2

目标系统

企业级 AI 至少
由哪些层组
成，为什么不
能只盯检索和
生成？

问题 3

课程主线

后续 14 周为什
么不是过度工
程化，而是在
补上线能力？



事故开场：Demo 能答，上线就炸

事故 1

规则漂移

症状

- 规则更新了，答案还在讲旧版本

本质原因

- 数据版本和更新链路没有正式进入系统

上线后果

- 错的不是一句回答，而是整个事实口径

事故 2

口径冲突

症状

- 同名术语跨部门口径不一致，系统却当成一个意思

本质原因

- 契约和语义层没有先写，检索只能硬拼相似度

上线后果

- 回答看起来流畅，业务却根本不敢采信

事故开场：Demo 能答，上线就炸

事故 3

权限越界

症状

- 低权限用户看到了高敏 SOP

本质原因

- 权限、PII、动作边界没有进入路线设计

上线后果

- 问题直接升级成合规和责任事故

事故 4

问题失踪

症状

- 坏例出现后根本没法复现

本质原因

- 数据、索引、Prompt、版本和 Trace 都不成链

上线后果

- 团队只能猜是哪一层漂了，优化变成碰运气



事故收口：这些炸点为什么都不是模型问题

事故收口

这些事故不是模型太弱，而是路线太短。

你如果把企业 AI 理解成“Prompt + 向量库 + LLM”，迟早会在真实数据、权限、工具调用和治理环节撞墙。

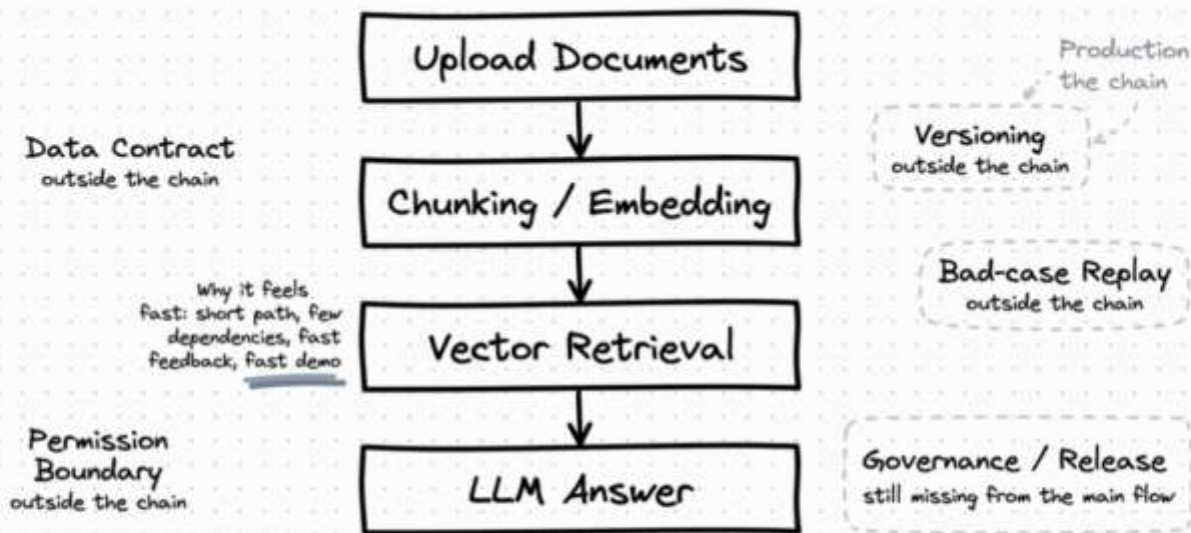


脚本式 RAG: 最短可用路径长什么样

最短路径图

"Script-style RAG: The Shortest Viable Path"

Fast to demo, but not yet production architecture



脚本式 RAG 之所以快，不是因为它已经完整，而是因为数据契约、版本、权限、回放和治理这些昂贵能力都还留在链外。

三条路线总览：你不是在选技术栈，而是在选后果

路线 1

脚本式 RAG

- 适合 PoC 和内部演示
- 先回答“值不值得做”
- 一进真实权限和动作链就会集中爆炸

路线 2

服务化 RAG

- 更稳定，有 API、缓存、基本日志
- 适合试点，不等于已经有统一底座
- 解决调用稳定性，不自动解决治理闭环

路线 3

企业级 AI 数据工程平台

- 目标不是大而全，而是可追溯、可治理、可回滚
- 不是 Day 1 做完全部，而是 Day 1 就知道最终要补齐到哪里



脚本式 RAG 为什么快，为什么两周后就开始脆

快在什么地方

- 路径短，依赖少，反馈快
- 最少的数据工程就能把“会答”做出来
- 现场 Demo 通常很顺
- 启动成本极低

脆在什么地方

- 复杂度被延迟支付
- 版本、权限、坏例回放、发布边界都不清楚
- 一旦数据换批、规则更新、工具链变化，就很难复盘
- 你以为缺模型技巧，实际缺工程链

关键判断

脚本式 RAG 的风险，不是效果差，而是它会让团队误以为“剩下只是再调一调模型”。

真正的目标对象：不是一个 RAG 系统，而是一条企业 AI 数据工程链

输入资产 → 检索生成 → 工具行为 → 治理发布

数据 把输入做成可追溯资产，而不是临时喂给模型的原料。

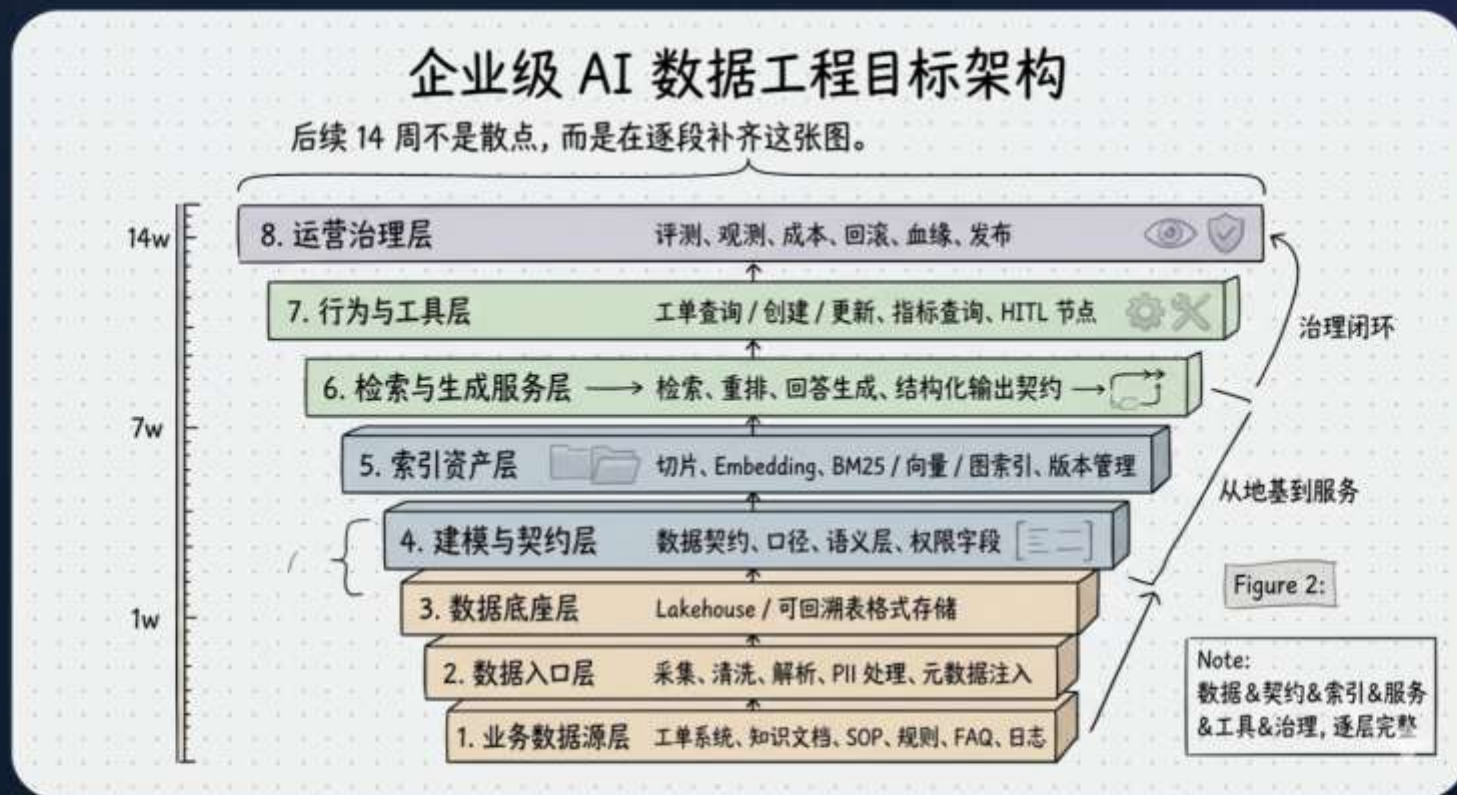
检索与生成 让回答不仅像样，还要带证据、能回归、可校验。

工具与行为 让系统从“会答”进入“能办”，但动作权限必须前置设计。

治理与发布 把评测、Tracing、版本、回滚和责任边界收成正式上线能力。

企业级 AI 数据工程目标架构

目标总图



后续 14 周不是散点技术，而是在逐层补齐这张企业级 AI 数据工程目标架构图。

八层目标架构：基础四层先决定你有没有事实土壤

1 数据源层

- 先知道系统到底建在什么数据之上
- 明确 owner、刷新频率、使用边界

2 数据入口层

- 先把解析、清洗、PII、元数据做对
- 不要让脏输入一路往后传

3 数据底座层

- 给 time travel、回放、回滚留地方
- 不要让旧数据互相覆盖

4 契约语义层

- 统一术语、字段、权限口径
- 让“冻结 / 限制 / 挂起”不再混成一个意思



八层目标架构：后四层决定你能不能真正负责

5 索引资产层

- 把索引做成可版本化资产
- 不要留一个黑盒向量库

6 检索生成层

- 让回答带证据、可校验、可回归
- 让问题能重放

7 行为工具层

- 从“会答”进入“能办”
- 把动作分级和 HITL 做成规则

8 治理发布层

- 把评测、Tracing、版本、回滚连起来
- 让责任边界真正可追



OmniSupport 映射：先看事实土壤

数据源 + 入口

先保输入

在项目里承担什么

- 接 FAQ、SOP、业务规则、工单数据
- 完成解析、清洗、PII 处理

缺了先爆什么

- 规则更新不一致，解析错误直接进索引

底座 + 契约

先保口径

在项目里承担什么

- 保留快照、统一术语、定义权限字段和业务口径

缺了先爆什么

- 同名术语冲突，旧版本无法回看，责任边界不清



OmniSupport 映射：再看检索和动作

索引 + 检索生成

先保证据

在项目里承担什么

- 管理 FAQ / SOP / 工单经验索引
- 输出带证据的结构化回答

缺了先爆什么

- 召回不可解释，证据不稳定，坏例难复现

工具行为

先保边界

在项目里承担什么

- 查询工单、建议建单、升级人工、审批节点

缺了先爆什么

- 从“会答”滑向“越权执行”

OmniSupport 映射：最后看治理发布

治理发布

先保可回滚

在项目里承担什么

- 评测、Tracing、变更记录、回滚、Runbook

缺了先爆什么

- 事故无法定位，试点永远不敢真正上线

项目定位

**OmniSupport 不是主线本身，
而是后续每一层能力的验证器。**

每学一层，都要能回到同一个业务世界里验证它为什么必要。



三阶段演进：MVP / 可试点 / 可上线

MVP → 可试点 → 可上线

阶段 1

MVP

先证明业务问题值得做，但别假装自己已经接近上线。

- 最小检索链路
- 最小证据引用
- 最小日志与坏例记录

阶段 2

可试点

让小范围真实用户开始使用，但要先知道哪些能力已经必须进场。

- 权限过滤
- 评测基线
- 可回归、可观测
- 基本降级策略

阶段 3

可上线

纳入正式业务流程，开始真正承担发布、回滚和审计责任。

- 版本、回滚、Runbook
- 评测与发布绑定
- 动作分级与 HITL
- SLO、Tracing、责任边界

Week02–Week15 不是加料，而是在补链

周次	真正在补哪一层	对应的上线问题
Week02	输入资产与契约	哪些数据配进系统，哪些输入必须带边界
Week03	采集与原始保留	为什么问题出现后必须能回看原始事实
Week04–08	湖仓、索引、检索与服务化	为什么索引、回答和证据要变成可交付资产
Week09–10	工具层与动作边界	为什么系统从“会答”进入“能办”时要先有权限与 HITL
Week11–15	评测、Tracing、治理与发布	为什么上线之后最贵的是回滚、责任边界和治理闭环



课堂收束：后面不是过度工程化，而是在补上线级 AI 的必要能力

本讲收束

**别再把企业 AI 理解成
“Prompt + 向量库 + LLM”。**

它真正要交付的是一条从输入、契约、索引、检索、行为到治理都能负责的工程链。
后面的课程，只是在逐层把这条链补完整。



项目能不能开工，先看基线和边界定没定

LESSON04 · 工程基线

别急着写 RAG：项目骨架、风险边界和落地蓝图，为什么第一周就得定？

这一讲不再问“仓库能不能先跑起来”，而是先回答：一个企业级 AI 项目，凭什么有资格开工。

Week01 要先把工程基线、风险边界和落地蓝图写清。后面 14 周的实现，才不会变成边做边返工。

E1042 冲突

项目全貌

基线链条

两份工件

一句话结论

本讲唯一主判断

**企业级 AI 项目最怕的，不是第一天效果差，
而是第一天就没有统一工程基线。**

没有这条基线，后面每一周做得越多，返工越大，责任边界也会越来越糊。

冲突开场：E1042 为什么不是一个“回答”问题

Demo 级系统通常会怎么做

- 去文档里检索 E1042、rollback、gateway upgrade
- 拼出一段看起来合理的处理建议
- 回答“建议先回滚，再检查依赖版本”
- 现场演示时，这个答案往往已经够像“会做事”

生产级系统还必须继续判断什么

- 当前客户的 SLA 等级和订阅权限是什么
- 这条产品线是否允许自动触发工单或回滚动作
- “回滚”是不是高风险动作，是否必须进 HITL
- 回答是否必须带证据锚点、trace_id、release_id
- 是否需要同步写入审计日志

冲突收口

企业 AI 的难点不在回答，而在负责。

Week01 要做的，不是先把功能写多，而是先把系统能负责到什么程度写清楚。



先看项目全貌：OmniSupport Copilot 到底是什么

它到底要交付什么

项目定义

- 面向企业支持场景，而不是开放域聊天
- 支持问答、证据引用、工单联动、人工介入
- 后面再补评测、Tracing、版本与回滚

它为什么不是只靠 PDF 问答

业务世界观

- 帮助中心、FAQ、Release Notes、API 文档、工单
- 安装手册、规格说明、接线图、故障排查视频
- 教学视频、录屏教程、错误码手册、社区问答



它凭什么能贯穿 15 周

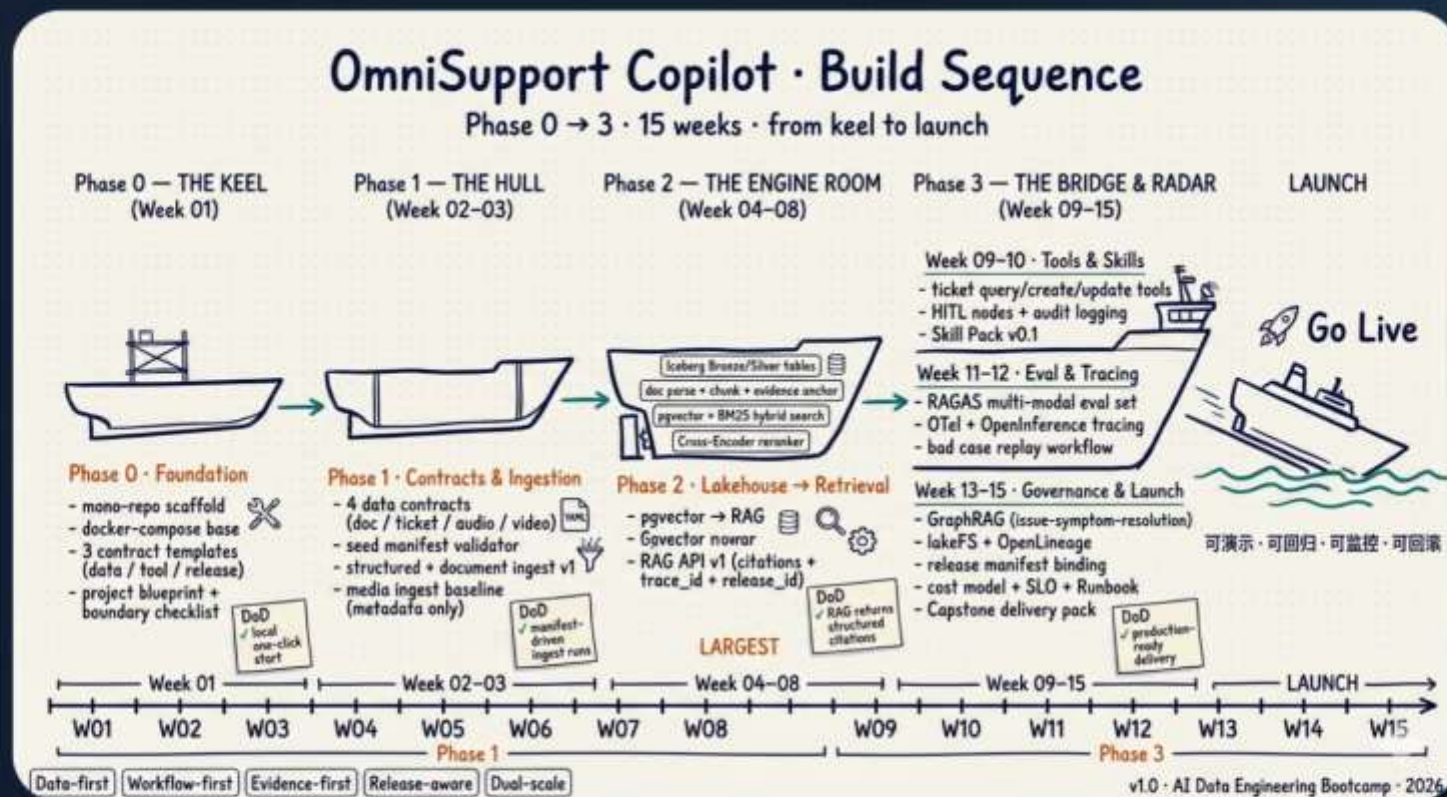
实施原则

Data-first / Workflow-first / Evidence-first / Release-aware / Dual-scale

Week01 先立统一蓝图、工程基线、风险边界和验收口径，后面 14 周再逐层补齐能力。

OmniSupport Copilot 的整体构建序列

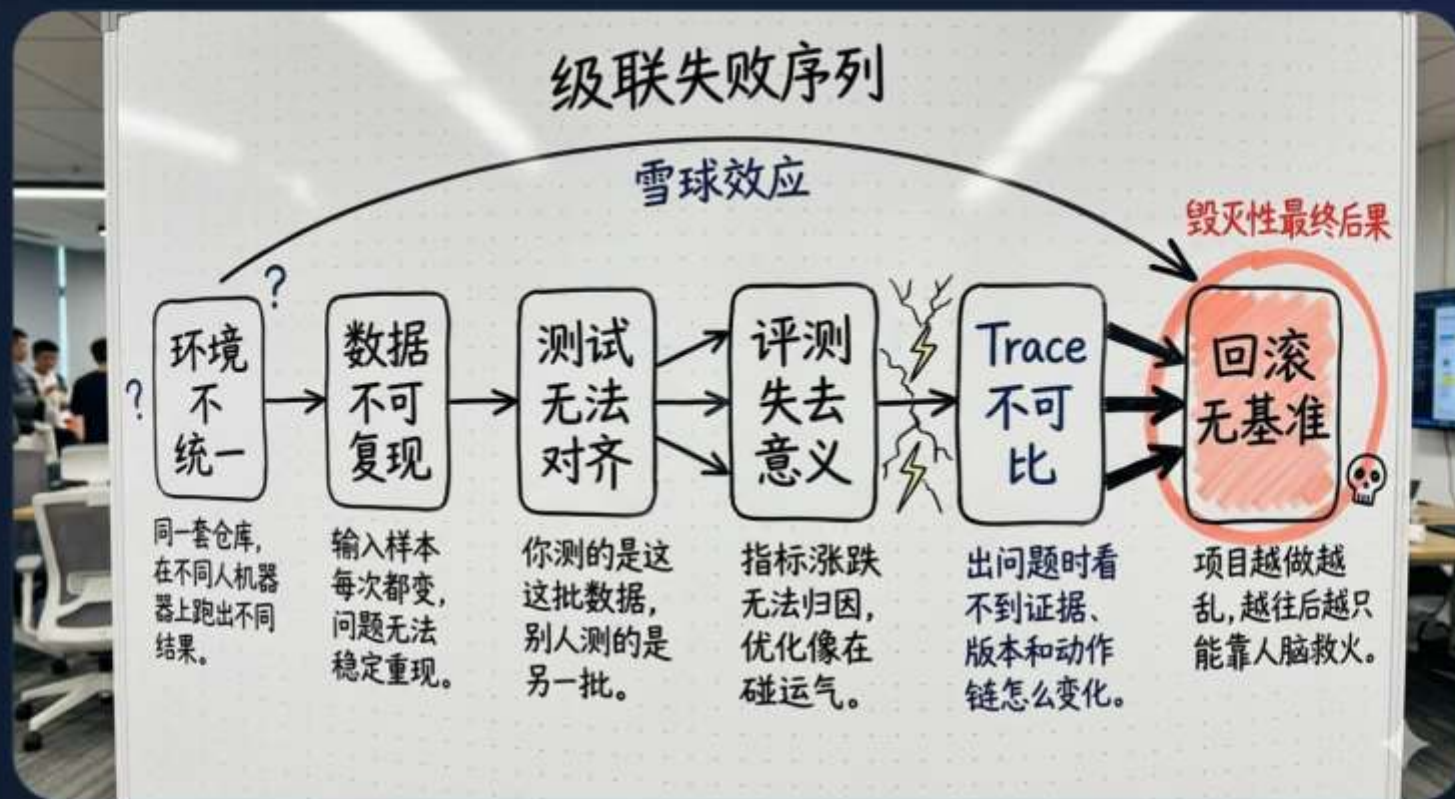
15 周构建路线图



Week01 不是孤立的开营课，而是 **Phase 0 - The Keel**：先把 mono-repo、compose 基线、contract 模板、项目蓝图和边界清单立住。

为什么 Week01 就要先定工程基线

级联失败序列



三类边界：工程基线真正要写的东西

三条边界图



工程基线不是一组目录，而是上线前必须先定义三条边界：系统边界、行为边界、工程边界。

项目骨架不是目录树，而是生产问题映射：先看底座四层

目录	在课程里的职责	它解决的生产问题
infra/	环境统一与基础依赖	为什么别人机器上跑不起来？
contracts/	输入输出与动作约束	哪些规则只是口头说的，没有被系统执行？
pipelines/	数据资产加工与编排	输入输出怎么可回放、可重试、可追溯？
services/	RAG 与工具服务化	能力如何暴露成稳定接口，而不是散落脚本？



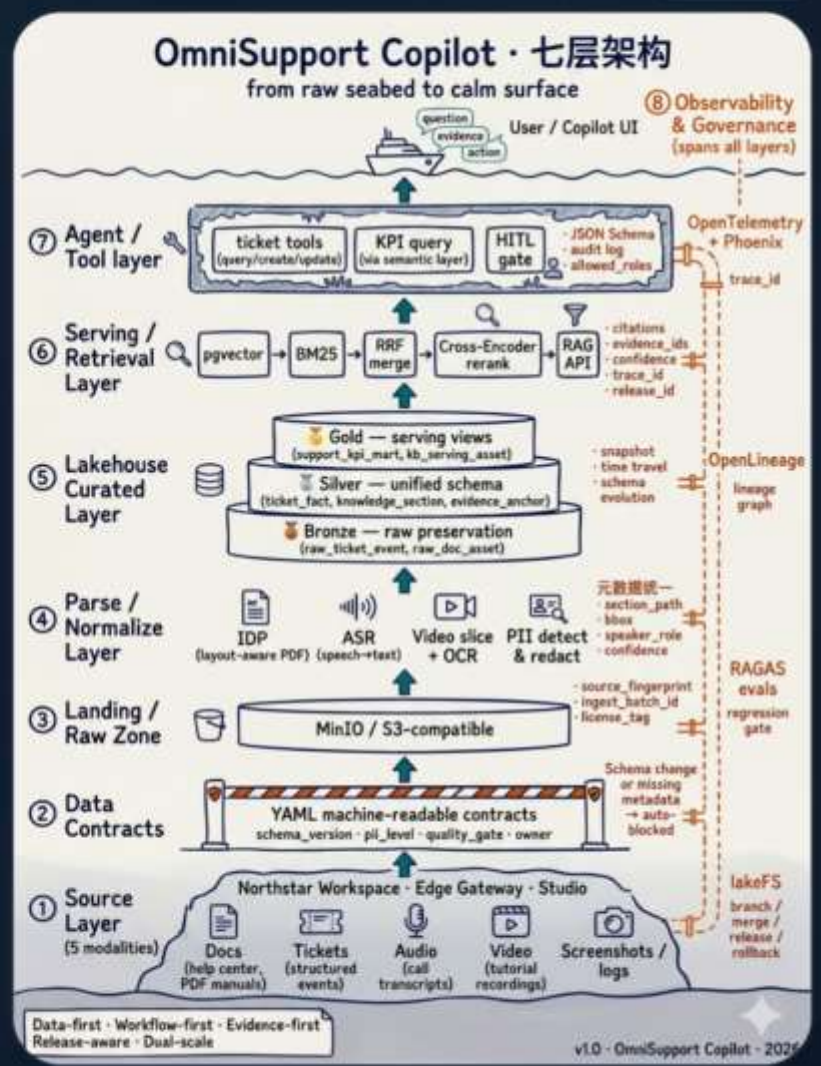
项目骨架不是目录树，而是生产问题映射：再看负责四层

目录	在课程里的职责	它解决的生产问题
<code>observability/</code>	Tracing / Phoenix / dashboards	出问题如何定位证据、版本、动作链？
<code>evals/</code>	评测集与回归机制	这次优化到底有没有真的变好？
<code>runbooks/</code>	运维与故障处理手册	事故发生后，谁按什么步骤恢复？
<code>blueprints/</code>	落地蓝图与共识文档	团队和后续实现凭什么对齐世界观？



七层架构：从输入到负责

七层总图



真正的企业 AI 系统不是只靠检索和生成两层支撑，而是从 source、contract、raw、parse、lakehouse、serving、agent 到 observability & governance 的完整工程链。

五步工程基线：前三步先把运行底座站住

01 配置隔离

`.env.example`

共享字段契约，机密不写死在代码里。

02 环境同构

用同一套
Compose 启动
服务、网络、
卷与依赖。

03 能力可验

先看关键能力
是否可达，而
不是只看进程
有没有启动。

这三步先站住

先把环境 and 能力跑成共同基线，再谈后面的复现、测试和上线。



五步工程基线：再补后两步，系统才可复现可检查

04 输入可复现

准备稳定种子数据，给测试、演示和回归共用。

05 边界可检查

契约测试先入场，让系统边界能被自动化检查。

一句话收口

Week01 的基线演示，本质上是在演示五个工程承诺。

配置隔离、环境同构、能力可验、输入可复现、边界可检查，缺一项，后面的协作和验收就没有共同基准。



风险边界：先把系统允许做什么说清楚

PII 分级

- P0 公共信息：可以直接使用
- P1 低敏业务信息：脱敏后使用
- P2 高敏客户信息：工具侧受控读取
- P3 强敏感信息：禁止进入模型上下文

动作边界

- 可执行：知识查询、工单状态查询
- 受限可执行：创建工单、更新低风险字段
- 高风险动作：回滚生产配置，必须进 HITL
- 禁止动作：越权查看敏感订阅、跳过证据直接下结论

HITL 不是系统失败，而是系统知道什么时候该停

HITL TRIGGERS

高风险操作 / 证据不足 / 状态冲突 / 权限不足

HITL 不是系统失败，而是系统知道什么时候不该自己做决定。真正的成熟，不是“什么都自动做”，而是该停的时候会停。

一句话判断

系统最危险的状态，不是它不会做事，而是它不知道什么时候应该把决定权交还给人。



风险边界：真正要管的是系统行为，而不是法务附录

01

PII 分级决定什么能进模型，什么必须停在模型外。

02

动作边界决定系统能查到哪一步、能做到哪一步。

03

HITL 负责在高风险、不确定和冲突时把决定权还给人。



Week01 要交的两份工件

工件 1 | 业务验收口径 & 风险边界清单

- 业务目标、目标用户、高价值场景、禁止场景
- PII 分级、动作分级、HITL 节点
- 失败与降级策略、验收口径

```
1 ## 业务验收口径 & 风险边界清单
2 - 业务目标:
3 - 目标用户:
4 - 禁止场景:
5 - HITL 节点:
6 - 验收口径:
```

工件 2 | AI 系统落地蓝图

- 一句话定义、业务世界观、核心对象
- 七层架构、技术选型理由、首周运行基线
- 后续 14 周演进路线

```
1 ## AI 系统落地蓝图 v1
2 - 一句话定义:
3 - 核心对象:
4 - 七层架构:
5 - 首周运行基线:
6 - 后续 14 周路线:
```

本讲小结：Week01 解决的是“有没有资格开工”

01

工程基线不是目录树，而是边界条件。

02

风险边界不是法务附录，而是系统行为开关。

03

落地蓝图不是汇报材料，而是后续实现与验收的共同基准。



THANKS

 极客时间 | 训练营